

Colegio Universitario de Cartago

Tecnologías de información

Investigación y desarrollo de una aplicación con React Native

Curso

TI-161 Programación V

Profesor

Ing. Gabriel González Solano

Grupo

01

Estudiantes

Aguilar Monge Rámses

Espinoza Méndez Fabián

Morales Rojas Alejandro

Morales Calderón José

Cartago, Costa Rica

III Cuatrimestre 2026

Introducción

Esta investigación aborda los fundamentos de React Native, su arquitectura y componentes, el consumo de APIs REST y la integración y validación de una aplicación. Los conceptos se aplican a un catálogo desarrollado con Expo que obtiene artículos públicos de DEV.to, como parte de la Tarea corta 3 de Programación V.

El trabajo relaciona los conceptos teóricos con la implementación del catálogo y analiza el recorrido de los datos desde la solicitud HTTP hasta su presentación en pantalla. Las pruebas automatizadas web y las exportaciones finalizaron correctamente; la ejecución en un dispositivo o emulador móvil continúa pendiente.

Repositorio del proyecto: Consultar el código fuente en GitHub

Fundamentos de React Native

Concepto de React Native

React Native es un framework de código abierto que permite desarrollar aplicaciones para Android e iOS con JavaScript y React. Fue creado en Facebook, hoy Meta. Su idea es compartir buena parte del trabajo entre plataformas, en lugar de mantener dos aplicaciones completamente separadas (React Native, 2026a).

La interfaz se construye con componentes. Podemos pensar en ellos como piezas que se combinan para formar una pantalla: un texto, una imagen, una tarjeta o un listado. En móvil, los componentes principales se apoyan en vistas nativas del sistema. Por eso, una aplicación con React Native no consiste simplemente en mostrar una página web dentro de una ventana (React Native, 2026a).

Compartir código tampoco significa que todo sea idéntico. Android e iOS tienen diferencias en permisos, apariencia e interacción. React Native permite atenderlas con código específico cuando hace falta (React Native, 2025c).

En este proyecto utilizamos Expo como conjunto de herramientas para trabajar con React Native. Expo facilita tareas como iniciar el servidor de desarrollo y abrir el proyecto en un dispositivo. No sustituye a React Native: nos ayuda a trabajar con él (Expo, 2026).

Historia y evolución

React Native surgió al llevar las ideas de React al desarrollo móvil. La intención era conservar una forma de construir interfaces mediante componentes y facilitar los ciclos de desarrollo, aprovechando las capacidades de cada plataforma (Occhino, 2015).

- Marzo de 2015: Facebook presentó públicamente React Native en su publicación sobre desarrollo móvil con técnicas de la web (Occhino, 2015).
- Septiembre de 2015: se anunció la versión para Android. La publicación también señala que React Native para iOS había sido presentado ese mismo año (Witte, 2015).
- Octubre de 2024: React Native 0.76 incorporó la Nueva Arquitectura de forma predeterminada. Este cambio renovó sistemas de comunicación y renderizado para mejorar la integración con las plataformas nativas y con funciones modernas de React (The React Team, 2024).
- Septiembre de 2025: Expo publicó el SDK 54 con React Native 0.81 y React 19.1. Esa es la familia de versiones que utiliza nuestra base actual (Expo, 2025).

Esta evolución ayuda a entender por qué los tutoriales de diferentes años pueden explicar React Native de manera distinta. Para este trabajo tomamos como referencia la versión del proyecto; no asumimos que sea la más reciente.

Comparación con Flutter y Xamarin y .NET MAUI

Las tres opciones buscan compartir trabajo entre plataformas, pero lo hacen con herramientas y enfoques distintos. La siguiente comparación resume sus diferencias; no representa una prueba de velocidad entre ellas.

React Native

Utiliza JavaScript o TypeScript y el modelo de componentes de React. En Android e iOS, sus componentes principales se apoyan en vistas nativas. Para un equipo que ya conoce JavaScript, esto permite aprovechar conocimientos previos, aunque todavía debe aprender las particularidades del desarrollo móvil (React Native, 2026a).

Flutter

Utiliza Dart y construye la interfaz mediante widgets. Cuenta con su propio sistema de renderizado para dibujar la mayoría de sus elementos, en lugar de basar cada widget en un control nativo del sistema. Permite compartir código entre móvil, web y escritorio, e integrarse con servicios de la plataforma cuando se necesita. Su enfoque resulta útil cuando se busca controlar la apariencia de la interfaz de forma consistente; aprender Dart y sus widgets forma parte del trabajo inicial (Flutter, 2026).

Xamarin y .NET MAUI

Es importante distinguirlos. Xamarin.Forms evolucionó hacia .NET MAUI, y el soporte de Xamarin terminó el 1 de mayo de 2024 (Microsoft, 2024).

.NET MAUI permite desarrollar interfaces con C# y XAML para Android, iOS, macOS y Windows desde una base compartida. También admite código específico por plataforma. Su relación con .NET puede ser conveniente para equipos que ya trabajan con C# y ese ecosistema (Microsoft, 2025).

Elección de React Native para el proyecto

La consigna pide crear la base con Expo y el repositorio ya utiliza JavaScript y React Native. Continuar con esa base permite aprovechar el trabajo de interfaz que ya existe y concentrarnos en los objetivos de la actividad. Esta es una decisión para nuestro contexto, no una afirmación de que una tecnología sea mejor en todos los casos.

Ventajas y desventajas

Ventajas

- Compartir una parte importante del código: podemos reutilizar componentes y lógica entre Android e iOS. El porcentaje depende de las necesidades de cada aplicación (React Native, 2025c).
- Aprovechar conocimientos de React y JavaScript: el modelo de componentes permite dividir una pantalla en piezas reutilizables (React Native, 2026a).
- Revisar cambios con rapidez: Fast Refresh permite ver muchas modificaciones durante el desarrollo sin reiniciar toda la aplicación; en algunos casos se necesita una recarga completa (React Native, 2026c).
- Trabajar con herramientas de apoyo: Expo ofrece un flujo para iniciar el servidor y abrir el proyecto en un dispositivo o emulador (Expo, 2026).

Desventajas y aspectos que debemos cuidar

- Las diferencias entre plataformas siguen existiendo. Algunos componentes, permisos o comportamientos requieren ajustes y comprobaciones propias de Android o iOS (React Native, 2025c).

- El rendimiento requiere atención. Un cálculo pesado o demasiados renderizados pueden afectar la respuesta de la interfaz. Utilizar React Native no elimina la necesidad de revisar cómo se comporta la aplicación (React Native, 2025a).
- Fast Refresh tiene límites. No todos los cambios conservan el estado y algunos provocan una recarga; es una ayuda para desarrollar, no una garantía de que todo funcione (React Native, 2026c).
- Hay que coordinar las versiones. En nuestra base, Expo SDK 54 está asociado con React Native 0.81 y React 19.1; cambiar dependencias por separado requiere revisar su compatibilidad (Expo, 2025).

Relación con el proyecto base

El proyecto se desarrolla sobre una base Expo con una interfaz de catálogo. La configuración inicial y la organización de carpetas permiten separar los componentes visuales, la lógica de consulta y los estilos, y facilitan la integración y verificación de la aplicación.

Durante la construcción de la interfaz se utilizaron datos de prueba. Posteriormente, el catálogo se conectó a una API REST y se verificaron la carga de artículos, su presentación y el manejo de errores. La estructura del proyecto y las comprobaciones realizadas se describen en la documentación de desarrollo.

Arquitectura componentes y Hooks en React Native

Arquitectura básica de una aplicación React Native

React Native es un framework de desarrollo móvil multiplataforma creado por Meta Platforms que permite construir aplicaciones nativas para Android e iOS utilizando JavaScript (o TypeScript) y la biblioteca React. A diferencia de los enfoques híbridos basados en WebViews (como Apache Cordova o Ionic en sus primeras versiones), React Native traduce la jerarquía de componentes declarada en React directamente en vistas nativas del sistema operativo anfitrión (React Native, 2025b).

Hilos principales de ejecución

El modelo de ejecución distribuye el trabajo de JavaScript, la presentación de la interfaz y el cálculo del diseño. Su organización depende de la arquitectura y del trabajo que se ejecuta; no constituye una regla universal de tres hilos fijos (React Native, 2023). Los siguientes conceptos permiten comprender estas responsabilidades:

- Hilo de JavaScript (JS Thread): ejecuta la lógica de la aplicación con el motor Hermes (por defecto), procesa el árbol de React, gestiona los estados (Hooks) y despacha las solicitudes de red.
- Hilo principal o nativo (Main / UI Thread): pinta la interfaz en pantalla y captura los eventos táctiles del usuario.
- Árbol de sombra y motor Yoga: representan la estructura de la interfaz y permiten calcular posiciones y dimensiones con Flexbox. El trabajo de diseño se coordina con el renderizado; no implica que toda arquitectura utilice un hilo de sombra independiente.

Evolución del Bridge a la Nueva Arquitectura

En su diseño clásico, React Native comunicaba el hilo de JavaScript y el hilo nativo mediante un puente asíncrono (Bridge) que serializaba mensajes en JSON. Este proceso generaba cuellos de botella al mover grandes volúmenes de datos o al desplazar listas extensas. Las versiones modernas implementan la Nueva Arquitectura (React Native, 2026b), compuesta por:

- JSI (JavaScript Interface): capa en C++ que reemplaza al Bridge; permite que JavaScript mantenga referencias directas a objetos nativos e invoque métodos sin serializar JSON.
- Fabric: nuevo sistema de renderizado compartido en C++, con soporte para renderizado concurrente y priorización de eventos urgentes.
- TurboModules: módulos nativos (cámara, sensores, red) que se cargan bajo demanda (lazy loading), reduciendo el tiempo de arranque.
- Codegen: genera código nativo tipado a partir de definiciones en JavaScript, garantizando seguridad de tipos entre capas.

Estructura y organización de componentes

En React Native la interfaz se construye con componentes funcionales: funciones independientes, reutilizables y declarativas que reciben propiedades inmutables (props), pueden administrar estado local y retornan una descripción visual en JSX (Boduch & Derks, 2020).

La interfaz se organiza como un árbol jerárquico con flujo de datos unidireccional (de arriba hacia abajo) (Eisenman, 2017):

- Componente raíz (App.js): punto de entrada que monta la pantalla principal.
- Pantallas o contenedores (src/screens/): coordinan el estado general y distribuyen la información. Ejemplo: HomeScreen.js.

- Componentes reutilizables (src/components/): reciben datos mediante props y notifican acciones con funciones callback. Ejemplos: ItemCard.js y CategoryFilter.js.
- Capa de datos y estilos (src/data/ y src/styles/): datos de prueba (mock data), funciones normalizadoras y constantes globales de diseño.

Estructura de carpetas utilizada en el proyecto:

```
proyecto/
|-- App.js                  # Punto de entrada
|-- src/
   |-- components/          # Componentes reutilizables
   |   |-- ItemCard.js      # Tarjeta de cada registro
   |   |-- CategoryFilter.js  # Barra horizontal de filtros
   |-- screens/             # Pantallas principales
   |   |-- HomeScreen.js    # Listado principal con FlatList
   |-- data/                # Datos de prueba y adaptadores
   |   |-- mockData.js
   |-- services/            # Consumo de APIs REST
   |-- styles/              # Constantes y estilos compartidos
      |-- theme.js
```

Componentes principales de React Native

React Native provee componentes nucleares (Core Components) que se traducen a controles nativos en cada plataforma (React Native, 2025b):

- View: contenedor básico para estructurar la interfaz (equivale a android.view.View en Android y UIView en iOS). Usa Flexbox con dirección vertical por defecto (flexDirection: 'column').
- Text: único componente válido para mostrar texto. Texto suelto fuera de un Text produce error en tiempo de ejecución. Admite numberOfLines para truncar contenido.
- Image: muestra imágenes locales (require) o remotas (source={{ uri: '...' }}). Las remotas requieren ancho y alto explícitos; resizeMode (cover, contain, stretch) controla el ajuste.
- ScrollView: contenedor desplazable que renderiza todos sus hijos en memoria al mismo tiempo. Es ideal para vistas cortas o barras horizontales de categorías (horizontal={true}).

• FlatList: componente de alto rendimiento para listas mediante virtualización. Mantiene una ventana de renderizado que puede incluir elementos visibles y cercanos a la zona visible. Sus propiedades esenciales incluyen data, renderItem, keyExtractor, ListHeaderComponent y ListEmptyComponent (React Native, 2026d).

• StyleSheet: API de estilos (StyleSheet.create({})) con sintaxis camelCase similar a CSS. Las dimensiones son independientes de la densidad de píxeles (dp en Android, pt en iOS).

Comparativa técnica entre ScrollView y FlatList:

Tabla 1

Comparación entre ScrollView y FlatList

Criterio técnico	**ScrollView**	**FlatList**
Estrategia de renderizado	Renderiza todos los hijos al montarse.	Virtualización: renderiza una ventana de elementos visibles y cercanos.
Consumo de memoria	Crece con la cantidad de elementos.	Se mantiene controlado aunque haya muchos registros.
Caso de uso en el proyecto	Barra de filtros por categoría (CategoryFilter.js).	Listado principal de tarjetas (HomeScreen.js).

Ejemplo integrador de los componentes principales:

```
import React from 'react';
import { View, Text, Image, FlatList, ScrollView, StyleSheet } from 'react-native';
const DATOS = [
  { id: '1', titulo: 'React Native', imagen: 'https://picsum.photos/200' },
  { id: '2', titulo: 'Expo SDK', imagen: 'https://picsum.photos/201' },
];
export default function EjemploComponentes() {
  return (
    <View style={styles.contenedor}>
      <ScrollView horizontal showsHorizontalScrollIndicator={false}>
        <Text style={styles.etiqueta}>Todos</Text>
        <Text style={styles.etiqueta}>Destacados</Text>
      </ScrollView>
```

```
      <FlatList
        data={DATOS}
        keyExtractor={(item) => item.id}
        renderItem={({ item }) => (
          <View style={styles.tarjeta}>
            <Image source={{ uri: item.imagen }} style={styles.imagen} />
            <Text style={styles.titulo}>{item.titulo}</Text>
          </View>
        )}
      />
    </View>
  );
}
const styles = StyleSheet.create({
  contenedor: { flex: 1, padding: 16, backgroundColor: '#F8FAFC' },
  etiqueta: { marginRight: 10, padding: 8, backgroundColor: '#DBEAFE', borderRadius: 8 },
  tarjeta: { flexDirection: 'row', alignItems: 'center', marginVertical: 8, padding: 12,
backgroundColor: '#FFF', borderRadius: 10 },
  imagen: { width: 50, height: 50, borderRadius: 25, marginRight: 12 },
  titulo: { fontSize: 16, fontWeight: 'bold', color: '#0F172A' },
});
```

Funcionamiento y ejemplos de useState y useEffect

Los Hooks son funciones de React que permiten usar estado y ciclo de vida en componentes funcionales (React, s. f.-a). Deben invocarse solo en el nivel superior del componente y nunca dentro de condicionales o bucles.

Gestión de estado local con useState

useState declara una variable de estado que se conserva entre renderizados. Devuelve el valor actual y una función para actualizarlo; al actualizarla, React vuelve a renderizar el componente. Sintaxis: const [estado, setEstado] = useState(valorInicial);

```
import React, { useState } from 'react';
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native';
export default function EjemploUseState() {
  const [favorito, setFavorito] = useState(false);
  return (
    <View style={styles.contenedor}>
      <Text style={styles.estado}>
        Estado: {favorito ? 'Guardado en Favoritos' : 'Sin guardar'}
      </Text>
```

```
      <TouchableOpacity style={styles.boton} onPress={() => setFavorito(!favorito)}>
        <Text style={styles.textoBoton}>Cambiar estado</Text>
      </TouchableOpacity>
    </View>
  );
}
const styles = StyleSheet.create({
  contenedor: { padding: 16, alignItems: 'center' },
  estado: { fontSize: 15, marginBottom: 10 },
  boton: { backgroundColor: '#2563EB', paddingVertical: 8, paddingHorizontal: 16,
borderRadius: 8 },
  textoBoton: { color: '#FFFFFF', fontWeight: 'bold' },
});
```

Efectos secundarios y ciclo de vida con useEffect

useEffect ejecuta efectos secundarios después del renderizado: cargar datos, realizar peticiones HTTP a una API REST, configurar temporizadores o suscribirse a eventos. Sintaxis:

```
useEffect(() => {
  // 1. Efecto secundario
  return () => {
    // 2. Limpieza (al desmontar o antes de volver a ejecutar)
  };
}, [dependencias]);
```

El comportamiento depende del arreglo de dependencias:

- [] (vacío): el efecto se ejecuta al montar el componente sin depender de valores reactivos externos. En desarrollo, Strict Mode puede realizar un ciclo adicional de preparación y limpieza; por ello, no debe asumirse una única ejecución en todas las condiciones (React, s. f.-b).
- [variable]: se ejecuta al montar y cada vez que la variable cambia.
- Sin arreglo: se ejecuta después de cada renderizado; debe evitarse si modifica el estado, porque puede generar un bucle infinito.

```
import React, { useState, useEffect } from 'react';
import { View, Text, FlatList, ActivityIndicator } from 'react-native';
const DATOS_PRUEBA = [
  { id: '1', nombre: 'Elemento Alfa' },
```

```
  { id: '2', nombre: 'Elemento Beta' },
];
export default function ListaConCicloDeVida() {
  const [datos, setDatos] = useState([]);
  const [cargando, setCargando] = useState(true);
  useEffect(() => {
    const temporizador = setTimeout(() => {
      setDatos(DATOS_PRUEBA);
      setCargando(false);
    }, 600);
    return () => clearTimeout(temporizador);
  }, []);
  if (cargando) {
    return <ActivityIndicator size="large" color="#2563EB" />;
  }
  return (
    <View style={{ flex: 1, padding: 16 }}>
      <FlatList
        data={datos}
        keyExtractor={(item) => item.id}
        renderItem={({ item }) => <Text>{item.nombre}</Text>}
      />
    </View>
  );
}
```

Consumo de APIs REST

Introducción

Las aplicaciones móviles suelen necesitar información que no se encuentra almacenada directamente en el dispositivo. Para obtenerla, pueden comunicarse con servicios externos mediante interfaces de programación de aplicaciones (API). Una API REST permite solicitar y manipular recursos a través de HTTP siguiendo convenciones que facilitan la comunicación entre sistemas.

En esta investigación se explican los fundamentos de las API REST, los métodos HTTP, el consumo de servicios con fetch() y Axios, la interpretación de JSON y la presentación de datos en React Native. Los conceptos se relacionan con el desarrollo práctico realizado para la Tarea Corta 3: una aplicación con Expo que consulta artículos públicos de DEV.to y los presenta en un catálogo.

El objetivo no es únicamente describir las herramientas, sino comprender el recorrido de los datos: el cliente realiza una solicitud, el servidor responde, la aplicación interpreta la información, administra sus estados y finalmente representa los resultados en la interfaz.

Concepto y funcionamiento de una API REST

Concepto de API

Una API (Application Programming Interface o interfaz de programación de aplicaciones) es un conjunto de reglas y mecanismos mediante los cuales un programa puede solicitar servicios o datos a otro programa. La API define qué operaciones están disponibles, qué información debe enviarse y qué tipo de respuesta puede recibirse.

Por ejemplo, una aplicación de catálogo no necesita conocer cómo DEV.to almacena internamente sus artículos. En su lugar, solicita los artículos a través de un endpoint documentado y recibe una representación de esos recursos.

Concepto de REST

REST (Representational State Transfer) es un estilo arquitectónico para diseñar sistemas distribuidos. En una API que sigue este estilo, la información se organiza como recursos identificables mediante URI o direcciones. El cliente interactúa con esos recursos utilizando operaciones HTTP y recibe representaciones, comúnmente en formato JSON.

REST no es un lenguaje de programación ni una biblioteca. Es un conjunto de restricciones de diseño. Entre las ideas centrales se encuentran la separación cliente-servidor, la comunicación sin estado (stateless), una interfaz uniforme y la posibilidad de aprovechar mecanismos de caché cuando corresponda.

- Cliente-servidor: la interfaz de usuario y el servidor tienen responsabilidades separadas. El cliente solicita información; el servidor procesa la solicitud y responde.
- Sin estado: cada solicitud debe contener la información necesaria para ser entendida. El servidor no debería depender de recordar el contexto de solicitudes anteriores para interpretar la petición actual.
- Interfaz uniforme: los recursos se identifican de forma consistente y se manipulan mediante operaciones conocidas.
- Representaciones: el servidor entrega una representación del recurso, por ejemplo un objeto JSON.
- Caché: ciertas respuestas pueden almacenarse temporalmente para reducir solicitudes y mejorar el rendimiento, de acuerdo con las reglas de caché HTTP.

Recursos URI y endpoints

Un recurso es cualquier elemento que el servicio expone, como un artículo, usuario, producto o comentario. Una URI identifica dónde se encuentra ese recurso. En el uso cotidiano, suele llamarse endpoint a la dirección concreta que el cliente consulta para ejecutar una operación.

GET https://dev.to/api/articles?tag=reactnative&per_page=30

En este endpoint se distinguen tres partes:

- Servidor: https://dev.to
- Ruta: /api/articles, que identifica la colección de artículos.
- Parámetros de consulta: tag=reactnative filtra por la etiqueta indicada y per_page=30 solicita hasta 30 elementos por página.

La documentación de Forem indica que GET /api/articles recupera artículos publicados, admite filtrado por etiqueta mediante tag y utiliza 30 elementos por página de forma predeterminada. El parámetro per_page permite definir el tamaño de página dentro de los límites documentados (Forem, s. f.).

Flujo general de una API REST

1. El cliente prepara la solicitud: define el endpoint, el método HTTP y, si corresponde, encabezados o cuerpo.
2. La solicitud viaja al servidor: se transmite mediante HTTP o HTTPS.
3. El servidor procesa la petición: valida los parámetros, consulta sus recursos y determina el resultado.
4. El servidor responde: devuelve un código de estado, encabezados y, normalmente, un cuerpo con datos.

5. El cliente interpreta la respuesta: comprueba el estado, convierte el cuerpo al formato esperado y actualiza la interfaz.

La separación permite que una misma API sea utilizada por diferentes clientes, por ejemplo una aplicación móvil, una página web u otro servicio.

Métodos HTTP y uso de GET

HTTP define métodos que expresan la acción que el cliente desea realizar sobre un recurso. En las API REST, los métodos más habituales son GET, POST, PUT, PATCH y DELETE. La operación concreta que permite cada endpoint depende del contrato publicado por el servidor.

Tabla 2

Métodos HTTP y sus usos habituales

Método	**Propósito habitual**	**Ejemplo conceptual**
GET	Consultar o recuperar un recurso. No se utiliza para solicitar cambios en el estado del servidor.	GET /api/articles
POST	Enviar datos para crear un recurso o iniciar un procesamiento.	POST /api/articles
PUT	Crear o reemplazar la representación de un recurso en una URI determinada.	PUT /api/users/15
PATCH	Aplicar una modificación parcial a un recurso.	PATCH /api/users/15
DELETE	Solicitar la eliminación de un recurso.	DELETE /api/users/15

GET es el método principal en esta implementación porque el catálogo necesita consultar artículos publicados, no crear ni modificar contenido en DEV.to. GET se considera un método

seguro: su semántica está orientada a recuperar información y no a producir cambios solicitados en el recurso (MDN contributors, 2026b).

Profundización en GET

Una solicitud GET identifica el recurso que se desea consultar. Los criterios de búsqueda o filtros pueden enviarse como parámetros de consulta en la URL. En el proyecto se utiliza la etiqueta reactnative y se solicita un máximo de 30 artículos.

```
const response = await fetch(
  'https://dev.to/api/articles?tag=reactnative&per_page=30',
  {
    method: 'GET',
    headers: {
      Accept: 'application/vnd.forem.api-v1+json'
    }
  }
);
```

El encabezado Accept expresa el formato de representación que el cliente espera recibir. La documentación de Forem recomienda el valor application/vnd.forem.api-v1+json para utilizar la versión 1 de su API. El endpoint público de artículos no requiere una clave API para esta consulta pública documentada (Forem, s. f.).

GET normalmente no lleva un cuerpo de solicitud para transmitir los criterios de consulta; estos se expresan en la URL y sus parámetros. La respuesta puede contener una colección JSON con los artículos que cumplen el filtro.

Códigos de estado HTTP relevantes

El código de estado permite conocer el resultado general de la solicitud. Los códigos se agrupan en cinco clases: 1xx informativos, 2xx satisfactorios, 3xx redirecciones, 4xx errores asociados a la solicitud del cliente y 5xx errores del servidor (MDN contributors, 2026a).

Tabla 3

Códigos de estado HTTP relevantes

Código	Significado	Interpretación práctica
200 OK	Solicitud exitosa.	Los artículos se recibieron correctamente.
201 Created	Recurso creado.	Respuesta frecuente después de una creación mediante POST.
204 No Content	Éxito sin cuerpo de respuesta.	La operación terminó, pero no hay contenido que mostrar.
400 Bad Request	Solicitud incorrecta.	Los parámetros enviados no cumplen lo esperado.
401 Unauthorized	Falta autenticación válida.	El servicio requiere credenciales.
403 Forbidden	Acceso denegado.	El cliente no tiene permiso para el recurso.
404 Not Found	Recurso no encontrado.	La ruta o el recurso solicitado no existe.
429 Too Many Requests	Demasiadas solicitudes.	El servidor limita temporalmente las peticiones.
500 Internal Server Error	Error interno del servidor.	El servicio encontró una condición inesperada.
503 Service Unavailable	Servicio no disponible.	El servidor no puede atender temporalmente la solicitud.

En las verificaciones del proyecto se observó una respuesta HTTP 429 durante consultas repetidas realizadas en la exploración. Este código no indica necesariamente un error del código cliente: informa que se superó el límite de solicitudes permitido durante un intervalo. Por ello, la aplicación debe mostrar un mensaje comprensible y evitar peticiones innecesarias.

Uso de Fetch y Axios

Fetch API

Fetch es una interfaz de JavaScript para realizar solicitudes de red y procesar sus respuestas. La función fetch() devuelve una Promesa (Promise), que permite trabajar de forma asíncrona. Al completarse la respuesta inicial, se obtiene un objeto Response con información

como el estado HTTP, los encabezados y métodos para leer el cuerpo (MDN contributors, 2025b).

Fetch está disponible como capacidad de la plataforma y no requiere instalar una dependencia adicional. En este proyecto se utiliza fetch() directamente, por lo que el consumo HTTP permanece separado de la interfaz y no se agrega una biblioteca externa.

Ejemplo básico con Fetch

```
async function obtenerArticulos() {
  try {
    const response = await fetch(
      'https://dev.to/api/articles?tag=reactnative&per_page=30'
    );
    if (!response.ok) {
      throw new Error(`Error HTTP: ${response.status}`);
    }
    const articulos = await response.json();
    return articulos;
  } catch (error) {
    console.error('No fue posible obtener los artículos:', error);
    throw error;
  }
}
```

La palabra async permite utilizar await dentro de la función. await espera la resolución de la Promesa sin bloquear la ejecución general de la aplicación. La propiedad response.ok es verdadera cuando el estado está en el rango 200–299. Es importante comprobarla porque fetch() no rechaza automáticamente la Promesa ante respuestas HTTP como 404 o 500; esos casos se reciben como objetos Response y deben validarse expresamente (MDN contributors, 2025b).

- response.status: devuelve el código numérico de estado HTTP, por ejemplo 200, 404 o 429.
- response.ok: indica si el estado está dentro del rango 200–299.
- response.json(): lee el cuerpo y lo interpreta como JSON. Es asíncrono y devuelve una Promesa.

El bloque try/catch permite manejar errores de red, errores lanzados al detectar estados HTTP no satisfactorios y fallos al interpretar el cuerpo. Una respuesta HTTP de error y una falla de conexión son situaciones distintas y conviene diferenciarlas.

Concepto de Axios

Axios es una biblioteca de JavaScript basada en Promesas para realizar solicitudes HTTP desde aplicaciones web y entornos compatibles con Node.js. Ofrece una API de alto nivel para configurar solicitudes, transformar datos, establecer valores predeterminados, utilizar interceptores y gestionar errores (Axios, s. f.).

```
import axios from 'axios';
async function obtenerArticulos() {
  const response = await axios.get(
    'https://dev.to/api/articles',
    {
      params: {
        tag: 'reactnative',
        per_page: 30
      },
      headers: {
        Accept: 'application/vnd.forem.api-v1+json'
      }
    }
  );
  return response.data;
}
```

En Axios, los parámetros de consulta pueden declararse mediante params y el contenido de la respuesta se encuentra normalmente en response.data. Axios también rechaza la Promesa por defecto ante respuestas que no cumplen su configuración de estados aceptados, lo que permite capturar muchos errores HTTP directamente en catch.

Comparación entre Fetch y Axios

Tabla 4

Comparación entre Fetch y Axios

Aspecto	**Fetch**	**Axios**
Disponibilidad	API integrada en plataformas compatibles; no requiere instalación adicional.	Biblioteca externa; debe agregarse al proyecto.
Lectura JSON	Se llama explícitamente response.json().	La respuesta JSON se transforma automáticamente en response.data en los casos habituales.
Errores HTTP	Se debe comprobar response.ok o response.status.	Rechaza por defecto estados fuera del rango aceptado.
Parámetros URL	Se pueden construir en la URL o con URLSearchParams.	Admite la opción params.
Configuración común	Opciones de fetch y objetos Request/Headers.	Instancias, configuración predeterminada e interceptores.
Uso en el proyecto	Es la opción seleccionada.	Se estudia como alternativa, pero no se instaló.

No existe una opción universalmente superior. Fetch es suficiente para solicitudes sencillas y reduce dependencias. Axios puede resultar conveniente cuando una aplicación necesita configuración compartida, interceptores o una API de consumo más uniforme. Para el catálogo de esta tarea, fetch() cubre los requisitos y mantiene el proyecto ligero.

Manejo e interpretación de respuestas JSON

Concepto de JSON

JSON (JavaScript Object Notation) es un formato de intercambio de datos basado en texto. Se utiliza para representar información estructurada que puede transmitirse entre sistemas. Aunque su sintaxis se parece a la de los objetos de JavaScript, JSON es un formato independiente y tiene reglas propias (MDN contributors, 2025a).

JSON admite cadenas, números, valores booleanos, null, arreglos y objetos. Los nombres de las propiedades deben escribirse como cadenas entre comillas dobles. No admite directamente funciones, comentarios ni valores undefined.

```
{
 "id": 123,
 "title": "Introducción a React Native",
 "description": "Conceptos para desarrollar aplicaciones móviles",
 "tag_list": ["reactnative", "javascript"],
 "reading_time_minutes": 5,
 "published": true,
 "cover_image": null
}
```

El ejemplo representa un artículo. Las llaves delimitan un objeto; cada propiedad tiene un nombre y un valor. tag_list es un arreglo de cadenas, reading_time_minutes es un número, published es booleano y cover_image contiene null para indicar ausencia de valor.

Interpretar JSON recibido

Cuando fetch() recibe una respuesta, response.json() lee el cuerpo y analiza el texto JSON para producir valores de JavaScript que el programa puede utilizar. Si el cuerpo no contiene JSON válido, la operación puede fallar y debe manejarse la excepción.

```
const response = await fetch(url);
if (!response.ok) {
  throw new Error(`HTTP ${response.status}`);
}
const data = await response.json();
if (!Array.isArray(data)) {
  throw new Error('La respuesta no tiene el formato esperado');
}
```

En el caso del catálogo, la respuesta esperada es un arreglo de artículos. Por eso, además de validar que la solicitud haya sido exitosa y que el JSON pueda analizarse, el servicio comprueba Array.isArray(payload). Esta validación evita tratar una estructura inesperada como si fuera una lista.

Validaciones necesarias

- Estado HTTP: verificar si la solicitud fue exitosa antes de procesar el cuerpo.
- Formato JSON: controlar errores de análisis cuando el cuerpo no sea JSON válido.
- Estructura: comprobar que el tipo de dato recibido coincida con lo esperado, por ejemplo un arreglo.
- Campos opcionales: no asumir que todos los artículos contienen imagen, descripción u otros campos.
- Lista vacía: un arreglo vacío puede ser una respuesta válida; no debe confundirse con un error.

La validación estructural es importante porque una respuesta puede ser JSON válido y aun así no contener la estructura que la aplicación necesita. Por ejemplo, un objeto JSON no es equivalente a un arreglo JSON.

Renderizado de información obtenida de una API

El estado en React

Una vez recibidos los datos, la interfaz debe conservarlos y actualizarse cuando cambien. React ofrece Hooks para administrar estado y sincronizar componentes con sistemas externos. useState permite declarar una variable de estado y una función para actualizarla; useEffect permite ejecutar efectos secundarios, como una solicitud de red, cuando el componente se monta o cuando cambian sus dependencias (React, s. f.-b, s. f.-c).

useState

```
const [apiData, setApiData] = useState([]);
const [apiLoading, setApiLoading] = useState(true);
const [apiError, setApiError] = useState(null);
```

En este ejemplo se mantienen tres estados independientes:

- apiData: contiene los artículos obtenidos.

• apiLoading: indica si la solicitud sigue en proceso.

• apiError: contiene un mensaje cuando ocurre un problema.

Cuando se llama a la función actualizadora, React programa una nueva representación del componente con el valor actualizado. De esta forma, los datos recibidos pueden reflejarse en la pantalla sin manipular manualmente el árbol visual.

useEffect para iniciar la solicitud

```
useEffect(() => {
  const controller = new AbortController();
  async function cargar() {
    setApiLoading(true);
    setApiError(null);
    try {
      const articulos = await fetchCatalogItems({
        signal: controller.signal
      });
      setApiData(articulos);
    } catch (error) {
      if (error.name !== 'AbortError') {
        setApiData([]);
        setApiError('No se pudieron cargar los artículos.');
      }
    } finally {
      setApiLoading(false);
    }
  }
  cargar();
  return () => controller.abort();
}, []);
```

El arreglo vacío de dependencias indica que el efecto no depende de valores reactivos externos y se ejecuta al establecerse el componente. El efecto devuelve una función de limpieza que cancela la solicitud si deja de ser necesaria. React recomienda utilizar la limpieza de los efectos para evitar que una operación externa permanezca activa cuando cambia el ciclo de vida del componente (React, s. f.-b).

El ejemplo es ilustrativo: el proyecto real separa esta lógica en el Hook useCatalogItems.js y el servicio catalogApi.js.

Estados de carga error y lista vacía

La interfaz debe representar de manera distinta las situaciones que puede experimentar el usuario:

Tabla 5

Estados de una solicitud y su presentación

Estado	Condición	Respuesta visual esperada
Cargando	La solicitud todavía no termina.	Indicador de actividad y texto de carga.
Éxito con datos	La solicitud terminó y el arreglo contiene artículos.	Listado de tarjetas.
Éxito sin datos	La solicitud terminó correctamente y el arreglo está vacío.	Mensaje de lista vacía.
Error	Falló la conexión, el servidor respondió con error o los datos no pudieron procesarse.	Mensaje de error comprensible.

Distinguir estos estados evita mensajes engañosos. Una lista vacía no necesariamente significa que hubo un error; puede ser el resultado correcto de una consulta sin coincidencias. Del mismo modo, un error de conexión no debe presentarse como si el servidor hubiera respondido con cero registros.

FlatList en React Native

FlatList es un componente de React Native diseñado para representar listas desplazables de manera eficiente. Recibe una colección mediante data y utiliza renderItem para indicar cómo representar cada elemento. También admite keyExtractor para obtener una clave estable, además de componentes para encabezados, separadores y contenido vacío.

```
<FlatList
  data={items}
  keyExtractor={(item) => String(item.id)}
  renderItem={({ item }) => (
    <ItemCard item={item} />
  )}
  ListEmptyComponent={renderEmptyList}
/>
```

En la aplicación, HomeScreen utiliza FlatList para presentar los artículos normalizados. Cada artículo se entrega al componente ItemCard, mientras que CategoryFilter permite seleccionar una categoría. FlatList facilita el renderizado de colecciones y evita tener que construir manualmente una lista de vistas.

La documentación de React Native señala que FlatList es un componente optimizado para listas planas y que keyExtractor permite definir la clave utilizada para identificar los elementos. También debe considerarse que FlatList es un componente de renderizado puro: los cambios en información externa que afecten la lista deben reflejarse en sus props (React Native, 2026d).

Aplicación de los conceptos en el proyecto

API seleccionada

Para el desarrollo práctico se seleccionó DEV.to, una plataforma de publicación de artículos tecnológicos que expone una API pública de Forem. La consulta implementada utiliza el endpoint GET /api/articles con los parámetros tag=reactnative y per_page=30. La documentación oficial describe este endpoint como una consulta de artículos publicados y documenta el filtrado por etiqueta y la paginación.

https://dev.to/api/articles?tag=reactnative&per_page=30

La aplicación envía el encabezado Accept con el tipo application/vnd.forem.api-v1+json, correspondiente a la versión 1 de la API de Forem. Durante la verificación funcional, la consulta devolvió 30 registros, que se mostraron en el catálogo.

Organización de responsabilidades por archivo

Tabla 6

Responsabilidades de los archivos del proyecto

Archivo	Responsabilidad
catalogApi.js	Realiza la petición GET con fetch(), valida response.ok, procesa JSON, comprueba la estructura y clasifica errores HTTP, de conexión o formato.
useCatalogItems.js	Administra apiData, apiLoading y apiError mediante useState. Ejecuta la carga con useEffect y cancela la solicitud con AbortController cuando corresponde.
normalizeItem.js	Adapta los campos de los artículos de DEV.to al modelo requerido por la tarjeta: id, title, subtitle, description, image, badge y meta.
App.js	Ejecuta el Hook y entrega apiData, apiLoading y apiError a HomeScreen mediante props.
HomeScreen.js	Administra la presentación, aplica filtros por categoría y muestra estados de carga, error o lista vacía. Conserva los componentes visuales reutilizables.

Recorrido de los datos

1. Solicitud: App ejecuta useCatalogItems y el Hook inicia la carga.
2. Servicio: fetchCatalogItems realiza la solicitud GET a DEV.to.
3. Respuesta: el servicio comprueba el estado HTTP, interpreta el cuerpo como JSON y valida que sea un arreglo.
4. Estado: el Hook guarda los artículos o el error y actualiza el indicador de carga.
5. Comunicación: App pasa los estados a HomeScreen mediante props.

6. Adaptación: HomeScreen aplica normalizeItem a cada artículo para obtener el modelo visual.
7. Renderizado: FlatList representa cada elemento mediante ItemCard y permite utilizar los filtros de categoría.

Adaptación del modelo

La API entrega propiedades propias de DEV.to, por ejemplo title, description, cover_image, social_image, tag_list y reading_time_minutes. La interfaz existente no debe depender directamente de todas esas propiedades. Por eso normalizeItem funciona como adaptador: traduce la estructura externa a una estructura uniforme que los componentes visuales ya conocen.

```
// Ejemplo simplificado de adaptación
{
  id: String(rawItem.id),
  title: rawItem.title,
  subtitle: getArticleCategory(rawItem.tag_list),
  description: rawItem.description,
  image: rawItem.cover_image || rawItem.social_image,
  badge: rawItem.tag_list[0]?.toUpperCase(),
  meta: `Lectura: ${rawItem.reading_time_minutes} min`
}
```

La adaptación desacopla la interfaz del proveedor de datos. Si en el futuro se cambia la API, el trabajo principal consistiría en ajustar el servicio y el adaptador, procurando mantener el contrato que consume la pantalla.

Verificaciones prácticas realizadas

Las verificaciones funcionales del catálogo permitieron observar los resultados de la siguiente tabla. Estas observaciones se complementan con las pruebas automatizadas descritas en el apartado de integración y validación.

Tabla 7

Resultados de las verificaciones funcionales

Verificación	Resultado observado
Solicitud GET y recepción de artículos	PASS: la aplicación mostró 30 registros de DEV.to.
Renderizado de artículos	PASS: se visualizaron títulos, descripciones, imágenes y tiempo de lectura.
Manejo de desconexión	PASS: al perder conexión se presentó un mensaje indicando que no fue posible conectar con la API.
Filtros por categoría	PASS: se confirmó el funcionamiento de los filtros.
Favoritos	PASS: se confirmó la interacción de marcar y desmarcar favoritos.
Exportación web de Expo	PASS: la exportación web compiló correctamente.
Exportación Android de Expo	PASS: la exportación Android compiló correctamente.

La validación automatizada posterior incluyó respuestas vacías y distintos errores controlados, como se documenta en el apartado de integración y validación. La ejecución en un dispositivo físico o emulador continúa pendiente y debe registrarse por separado.

Conclusiones

Las API REST permiten que una aplicación móvil obtenga información de servicios externos mediante operaciones HTTP y recursos identificados por endpoints. Esta separación hace posible que el cliente se concentre en la experiencia de usuario mientras el servidor administra y proporciona los datos.

El método GET es apropiado cuando la aplicación necesita consultar información sin solicitar modificaciones en el servidor. Para consumirlo desde JavaScript, fetch() ofrece una alternativa integrada y basada en Promesas; Axios constituye otra opción mediante una biblioteca externa con utilidades adicionales.

JSON facilita el intercambio de información estructurada. Sin embargo, no basta con convertir una respuesta a JSON: también es necesario validar el estado HTTP, comprobar la estructura recibida y contemplar campos opcionales, respuestas vacías y errores.

En React Native, useState permite conservar los datos y estados de la solicitud, mientras que useEffect permite iniciar y limpiar el efecto asociado a la comunicación externa. FlatList facilita presentar colecciones de registros. La separación entre servicio, Hook, adaptador y pantalla mejora la organización y permite que la interfaz no dependa directamente del proveedor de datos.

En el proyecto de la Tarea corta 3, estos conceptos se aplicaron mediante una consulta pública a DEV.to. Los artículos recibidos se adaptan al modelo visual y se muestran en el catálogo mediante componentes reutilizables. Las verificaciones funcionales confirmaron la carga de artículos, la presentación, los filtros, los favoritos y el manejo de desconexión.

Integración validación y documentación

La integración reúne la base Expo, los componentes visuales y el consumo de artículos de DEV.to en una aplicación. En esta etapa se comprobó el recorrido completo en navegador, se corrigió el desbordamiento del contador en pantallas pequeñas y se preparó la documentación de ejecución y entrega. La aplicación pasó doce pruebas del servicio y nueve escenarios web; la validación en un dispositivo móvil sigue pendiente.

Flujo completo de la aplicación

El punto de entrada es App.js. Este componente ejecuta useCatalogItems y entrega a HomeScreen los estados apiData, apiLoading y apiError mediante props. De esta manera, la pantalla recibe la información que necesita sin realizar directamente la petición HTTP.

El Hook inicia una operación asíncrona en useEffect. Primero activa la carga y limpia el error anterior; después llama a fetchCatalogItems. El servicio solicita artículos publicados con la etiqueta reactnative y un máximo de treinta resultados por página. El endpoint público utilizado es https://dev.to/api/articles?tag=reactnative&per_page=30. La documentación de Forem describe la consulta de artículos y sus parámetros de filtrado y paginación (Forem, s. f.).

El servicio comprueba response.ok, interpreta el cuerpo como JSON y valida que sea un arreglo. Si todo es correcto, el Hook guarda los artículos. Si ocurre un fallo, clasifica el resultado como error de red, HTTP o formato y prepara un mensaje para la interfaz. En ambos casos termina el estado de carga.

HomeScreen transforma cada artículo mediante normalizeItem. Este adaptador convierte los campos externos al contrato de ItemCard: identificador, título, categoría, descripción, imagen, insignia y metadatos. FlatList recibe los registros filtrados y representa las tarjetas. Las

acciones de CategoryFilter y los botones de favoritos actualizan el estado local y provocan una nueva representación de la pantalla.

El recorrido puede resumirse así: App → Hook → servicio HTTP → respuesta JSON → estado → adaptador → FlatList → ItemCard. Las acciones del usuario vuelven al estado local mediante callbacks. No existe un servidor propio ni una base de datos de favoritos en esta entrega.

Relación entre componentes Hooks y servicios

La separación de responsabilidades facilita la integración. catalogApi.js conoce el endpoint y las validaciones de transporte. useCatalogItems.js conoce el ciclo de la solicitud. HomeScreen conoce los estados visuales y las interacciones. ItemCard y CategoryFilter reciben datos y funciones mediante props. Los estilos compartidos se mantienen en theme.js.

useState permite conservar los valores que determinan lo que se presenta. El Hook mantiene los estados de la solicitud; la pantalla conserva la categoría y los identificadores favoritos. No se modifica directamente un arreglo de estado: el cambio de favorito utiliza la función actualizadora y construye un nuevo arreglo a partir del anterior.

useEffect sincroniza el componente con un sistema externo, en este caso una petición de red. La limpieza del Hook cancela la solicitud con AbortController y evita actualizar el estado después del desmontaje mediante la variable isMounted. La documentación de React establece que la limpieza se ejecuta antes de repetir un efecto con dependencias diferentes y al retirar el componente (React, s. f.-b).

Un arreglo de dependencias vacío expresa que el efecto no depende de valores reactivos externos; no debe entenderse como garantía universal de una única ejecución. React puede

efectuar un ciclo adicional de preparación y limpieza durante el desarrollo con Strict Mode. Por eso la limpieza debe ser correcta aunque la aplicación haga más de una preparación del efecto.

Ejecución mediante Expo

La base utiliza Expo SDK 54, React Native 0.81.5 y React 19.1.0. Para reproducirla se instala Node.js 20.19.4 o superior, se abre la carpeta raíz del proyecto y se ejecuta npm ci. El archivo package-lock.json conserva las versiones resueltas; esto ayuda a instalar el mismo conjunto de dependencias.

El comando npm start inicia el servidor de desarrollo de Expo. Su terminal ofrece el QR y las opciones para abrir el proyecto. npm run web permite trabajar en navegador. En un teléfono se necesita un entorno compatible con la versión de SDK utilizada; la computadora y el dispositivo deben poder comunicarse. La documentación oficial de Expo describe este flujo de inicio y apertura (Expo, 2026).

npm run export genera los recursos para web y los paquetes de JavaScript/Hermes de Android e iOS. Esta comprobación valida el empaquetado y la resolución de imports, pero no equivale a instalar un APK ni ejecutar una aplicación iOS. Se debe distinguir entre compilar los recursos y probar la experiencia en cada plataforma.

Estrategia de validación

Se utilizaron dos niveles de prueba. Las pruebas del servicio sustituyen fetch con respuestas controladas para verificar resultados y clasificación de errores sin depender de Internet. Las pruebas de navegador abren la aplicación exportada y comprueban lo que ve el usuario, además de una consulta real a DEV.to. Esta combinación permite separar problemas de lógica, presentación y disponibilidad del servicio externo.

La documentación de React Native distingue pruebas de lógica, componentes e integración de extremo a extremo. También advierte que las comprobaciones en JavaScript no sustituyen la evaluación de las partes nativas. Por esa razón, el alcance de esta entrega se declara como validación automatizada web y exportación móvil; la ejecución en dispositivo se registra como pendiente (Novak, 2026).

Las respuestas controladas cubren éxito, demora, lista vacía, errores HTTP, interrupción de red, JSON inválido y estructura inesperada. La prueba de demora conserva la solicitud pendiente durante más de 500 milisegundos para comprobar que la pantalla no sustituye la API por los datos de demostración. El caso de HTTP 500 verifica también que no aparezca el mensaje de lista vacía.

La prueba de interacciones utiliza dos artículos conocidos. Comprueba que el contador de favoritos cambia de cero a uno y vuelve a cero; también verifica que el filtro reduce el listado de dos artículos a uno y que Todos restaura ambos. Las imágenes de ese caso son controladas para que la prueba no dependa de un proveedor externo.

Resultados e incidencia corregida

La ejecución del 5 de noviembre de 2026 produjo doce pruebas de servicio aprobadas y nueve escenarios de navegador aprobados, sin errores JavaScript no capturados durante esos recorridos. Expo Doctor aprobó dieciocho comprobaciones; la validación de dependencias y la exportación de Android, iOS y web finalizaron correctamente. El caso real obtuvo HTTP 200 y treinta artículos, y verificó el primer título y la primera imagen.

Durante la primera ejecución funcional se detectó que el contador quedaba parcialmente fuera de pantalla a 320 píxeles de ancho. El encabezado distribuía el título y el contador en una fila sin salto. Se permitió el salto de línea y se hizo flexible el bloque del título. El mismo caso

que detectó el problema aprobó después del ajuste. Este resultado documenta una corrección respaldada por una prueba de regresión.

La documentación de desarrollo incluye las capturas, la matriz de pruebas y los pasos para la validación móvil. El archivo resultados-web.json conserva el momento de ejecución, los casos, su resultado y los errores capturados. Las capturas con artículos ficticios se identifican como respuestas controladas; la captura 09-api-real.png corresponde al servicio real.

Buenas prácticas de integración y documentación

La integración se preparó sobre versiones identificables del proyecto y se verificó sin conflictos. El registro técnico permite relacionar las comprobaciones con el código evaluado y documentar la corrección realizada durante la validación.

El README explica los requisitos, la ubicación desde la que se ejecuta cada comando, el recorrido de los datos, las pruebas y los límites. Esta información permite que otro integrante reproduzca el proyecto sin depender de instrucciones verbales. Los escenarios automatizados y sus datos controlados forman parte de la copia local del proyecto para poder repetir la verificación después de cambios.

Los ejemplos de estructura y componentes ilustran la organización de la aplicación. Para reproducir la versión integrada se debe consultar el árbol de carpetas y las instrucciones del README. La documentación técnica reúne la configuración, el funcionamiento y los resultados de las pruebas.

La validación debe considerar las características de ejecución de React Native. FlatList utiliza una ventana de renderizado que puede incluir elementos fuera de la zona visible, y la distribución del trabajo entre hilos depende de la arquitectura. Estas características deben tenerse en cuenta al evaluar el rendimiento y la respuesta de la interfaz (React Native, 2023, 2026d).

Límites y cierre de la entrega

Los favoritos se almacenan en memoria y se pierden al recargar. El catálogo consulta una página de hasta treinta artículos. No se implementaron paginación, persistencia ni un botón de reintento. Las imágenes requieren conexión y su disponibilidad depende de servicios externos. La prueba real comprueba la primera imagen, no garantiza la disponibilidad permanente de todas.

Antes de la entrega académica se debe ejecutar el recorrido en un dispositivo o emulador compatible y registrar su modelo, sistema operativo, fecha y evidencia. Deben comprobarse especialmente el desplazamiento, los controles táctiles y las zonas ocupadas por las barras del sistema. También debe revisarse el documento final y confirmar la entrega en CUC Virtual según el cronograma del trabajo.

La integración validada demuestra que los componentes, el estado y el servicio REST trabajan juntos en la versión web. La documentación y las pruebas dejan una base reproducible para terminar la verificación móvil y presentar el trabajo con resultados trazables.

Referencias

Axios. (s. f.). *First steps*. https://axios.rest/pages/getting-started/first-steps

Boduch, A., & Derks, R. (2020). *React and React Native: A complete hands-on guide to modern web and mobile development with React.js* (3.ª ed.). Packt Publishing.

Eisenman, B. (2017). *Learning React Native: Building native mobile apps with JavaScript* (2.ª ed.). O'Reilly Media.

Expo. (2025, 10 de septiembre). *Expo SDK 54*. https://expo.dev/sdk/54

Expo. (2026, 22 de septiembre). *Start developing*. https://docs.expo.dev/get-started/start-developing/

Flutter. (2026, 24 de octubre). *Flutter architectural overview*. https://docs.flutter.dev/resources/architectural-overview

Forem. (s. f.). *Forem API V1*. https://developers.forem.com/api/v1

MDN contributors. (2025a, 11 de julio). *JSON*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Glossary/JSON

MDN contributors. (2025b, 20 de agosto). *Using the Fetch API*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

MDN contributors. (2026a, 17 de septiembre). *HTTP response status codes*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status

MDN contributors. (2026b, 23 de septiembre). *HTTP request methods*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods

Microsoft. (2024, 1 de mayo). *Xamarin support policy*. https://dotnet.microsoft.com/en-us/platform/support/policy/xamarin

Microsoft. (2025, 4 de febrero). *What is .NET MAUI?* Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/maui/what-is-maui

Novak, V. (2026, 27 de octubre). *Testing*. React Native. https://reactnative.dev/docs/testing-overview

Occhino, T. (2015, 26 de marzo). *React Native: Bringing modern web techniques to mobile*. React Native. https://reactnative.dev/blog/2015/03/26/react-native-bringing-modern-web-techniques-to-mobile

React. (s. f.-a). *Built-in React Hooks*. https://react.dev/reference/react/hooks

React. (s. f.-b). *useEffect*. https://react.dev/reference/react/useEffect

React. (s. f.-c). *useState*. https://react.dev/reference/react/useState

React Native. (2023, 8 de noviembre). *Threading model*. https://reactnative.dev/architecture/threading-model

React Native. (2025a, 12 de agosto). *Performance overview*. https://reactnative.dev/docs/0.81/performance

React Native. (2025b, 19 de agosto). *Core components and APIs*. https://reactnative.dev/docs/0.81/components-and-apis

React Native. (2025c, 20 de agosto). *Platform-specific code*. https://reactnative.dev/docs/0.81/platform-specific-code

React Native. (2026a, 3 de junio). *Core components and native components*. https://reactnative.dev/docs/0.81/intro-react-native-components

React Native. (2026b, 14 de octubre). *About the New Architecture*. https://reactnative.dev/architecture/landing-page

React Native. (2026c, 19 de octubre). *Fast Refresh*. https://reactnative.dev/docs/0.81/fast-refresh

React Native. (2026d, 19 de octubre). *FlatList*. https://reactnative.dev/docs/0.81/flatlist

The React Team. (2024, 23 de octubre). *New Architecture is here*. React Native. https://reactnative.dev/blog/2024/10/23/the-new-architecture-is-here

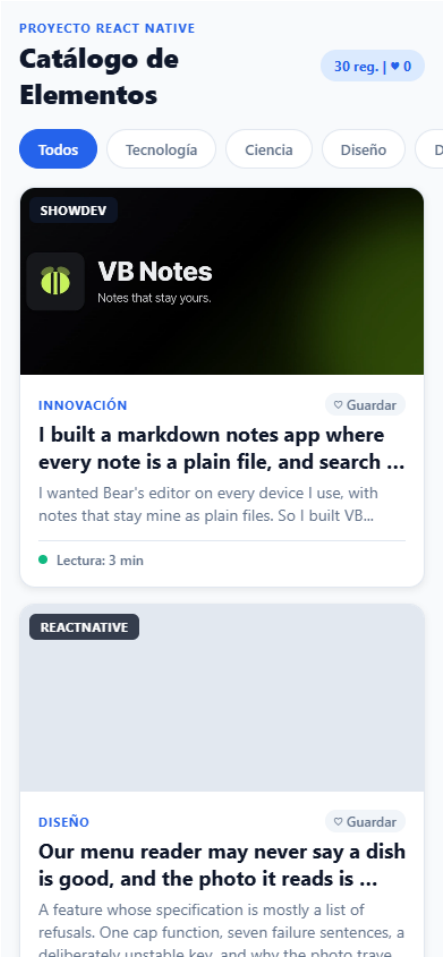
Witte, D. (2015, 14 de septiembre). *React Native for Android: How we built the first cross-platform React Native app*. React Native. https://reactnative.dev/blog/2015/09/14/react-native-for-android.html

Apéndice

Evidencias de las pruebas de integración

Figura A1

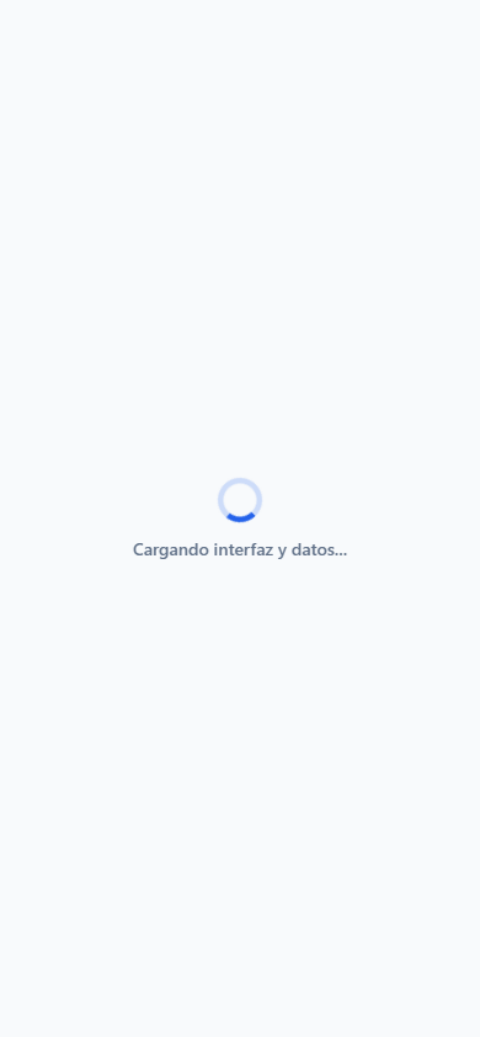
Catálogo con artículos obtenidos de DEV.to



Nota. Captura de la prueba web con respuesta HTTP 200 y 30 artículos. Se verificaron el primer título y la primera imagen. El tamaño de la ventana fue de 390 × 844 píxeles. Captura propia de la ejecución del 5 de noviembre de 2026.

Figura A2

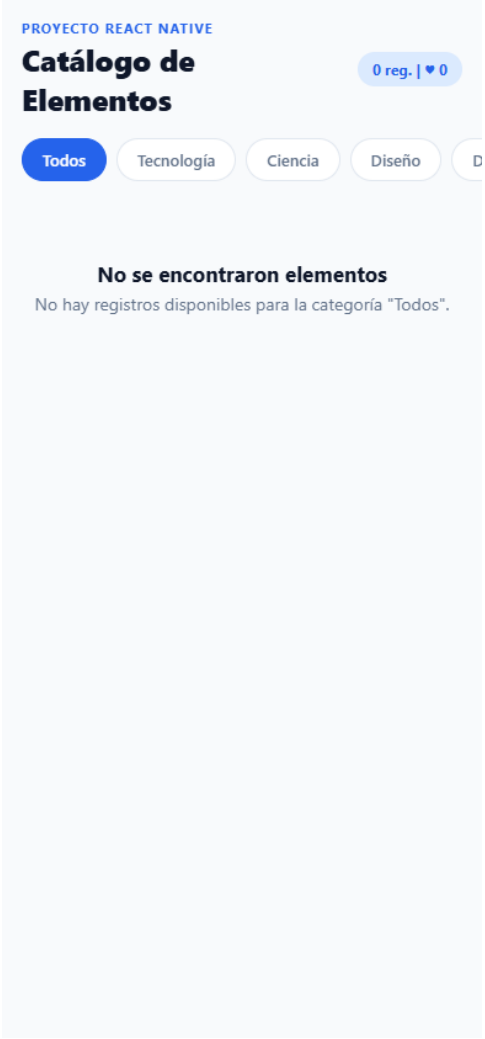
Indicador durante una solicitud pendiente



Nota. Prueba con respuesta controlada retenida durante más de 500 milisegundos. La interfaz mantiene el indicador de carga sin sustituir los resultados por datos de demostración. Captura propia de la ejecución del 5 de noviembre de 2026.

Figura A3

Presentación de una respuesta sin registros



Nota. Prueba con respuesta controlada HTTP 200 y un arreglo vacío. La pantalla muestra cero registros y un mensaje de lista vacía. Captura propia de la ejecución del 5 de noviembre de 2026.

Figura A4

Presentación de un error del servicio



Nota. Prueba con respuesta controlada HTTP 500. Se muestra un mensaje de error y se evita presentar el fallo como una consulta sin registros. Captura propia de la ejecución del 5 de noviembre de 2026.